

ZONKE REDEVELOPMENT PROJECT

TEAM 4 — BACKEND & DATABASE

Field	Details
System Developers	Kyle Jason Khoza; Kamogelo Moruri
Role	Backend & Database Developers
Main Responsibility	Build the data layer and server-side functionality required by the new Zonke prototype.
Tools / Technologies	VS Code; Git; GitHub; selected backend runtime/framework; selected database; Postman or a similar API testing tool.

1. What You Must Do

Task	Instruction
1. Choose stack	Select a backend technology and database that the team understands and can explain.
2. Set up project	Create the backend project, environment configuration and basic folder structure.
3. Identify entities	Identify the main data entities needed for the prototype, such as User, Client, Freelancer, Job, Proposal, Skill and Project where applicable.
4. Design database	Create the database structure, relationships, keys and important fields needed by the prototype.
5. Build backend structure	Create the server-side structure needed to process requests and work with the database.
6. Create API endpoints	Develop the priority endpoints required by the frontend, based on agreed requirements.
7. Test endpoints	Use Postman or another suitable tool to test requests, responses, validation and error handling.
8. Document integration	Give Team 3 the endpoint information, request formats and response structures needed for integration.

2. Work in This Order

Step	What to do
1. Decide	Choose a practical backend and database stack.
2. Model	Identify entities and design the data model.
3. Build	Create the backend and database structure.
4. API	Develop the priority endpoints.
5. Test	Test endpoints and fix issues.
6. Handoff	Provide clear integration information to Team 3.

3. Computer and Project Setup

Item	Instruction
Computer	Use a reliable computer with enough storage for the development tools, project files and Git repository. Keep the operating system and browser updated.
Code editor	Install VS Code or another editor you already understand. Make sure extensions required for your chosen technology are installed.
Git	Install Git and configure your name and email. Confirm that you can clone, pull, commit and push to the project repository.
GitHub	Use the shared Zonke repository. Work on your own branch where the team workflow requires it, pull before starting work, and commit regularly.
Browser	Use a modern browser such as Chrome, Edge or Firefox. Keep developer tools available for inspecting and testing the application.
Project folder	Create a clear local project folder. Do not keep the main project only on the desktop or in random folders.

Item	Instruction
Dependencies	Install only the packages, libraries, frameworks or tools actually required by the chosen technology. Record important setup steps so another System Developer can reproduce the environment.
Technology choice	Use technologies that the team understands and can explain. Do not choose a framework simply because it is popular or because AI suggested it.
Current Zonke site	Use the existing Zonke website as a reference for understanding the business, content and user journeys. The new application is being redeveloped from scratch.

4. Team-Specific Setup / Guidance

Area	Instruction
Project repository	All five teams contribute to the same Zonke development project. Do not create separate unrelated versions of the application.
Team communication	Keep the other teams informed about decisions, dependencies, changes to requirements and anything that may affect their work.
Use of AI	AI can help with explanations, ideas and troubleshooting, but it must not replace your understanding. Check, test and adapt any AI-generated material before using it.
Do not change business decisions	Do not invent or change the business model, target market, payment approach, user-data requirements, domain or hosting decisions without confirmation from the project/CTO direction.
Database setup	Install the selected database locally or use the approved development environment. Keep database setup instructions clear enough for another System Developer to reproduce.
Environment variables	Do not commit passwords, API keys, database passwords or other secrets to GitHub. Use environment variables or the approved configuration method.
API testing	Test endpoints independently before asking Team 3 to integrate them.
Prototype scope	Do not automatically build advanced functions such as payments, real-time messaging or complex authentication unless they are part of the agreed prototype requirements.
Data design	Use sensible names, appropriate data types, keys and relationships. Avoid creating duplicate data structures when a proper relationship can be used.

5. By Friday

Required Output	What should be available
Backend project	A running backend project with a clear structure.
Database model	Initial database/data model for the agreed prototype.
Priority endpoints	Working API endpoints for the functions required first.
API evidence	Test results showing successful and unsuccessful endpoint tests.
Integration information	Clear endpoint and data-format information for Team 3.

6. Working Rules

Rule	Instruction
GitHub	Commit regularly. Commit messages must describe the work completed. Each System Developer must be able to show their own contribution.
Communication	Tell the other teams when work is ready, blocked or dependent on another team's work.
Evidence	Keep useful evidence such as screenshots, designs, code commits, test results, diagrams or API results. Evidence should prove what was actually done.
Understanding	Do not copy and paste work from AI without understanding it. You must be able to explain your code, design decisions, tests and recommendations.
Professional practice	Keep files organised, use clear names, avoid unnecessary duplication, and ask for help when you are genuinely blocked.
Build from scratch	The new Zonke application is a fresh redevelopment. The existing website is a reference point, not a codebase that must be copied.